

BIT4107, MOBILE APPLICATION DEVELOPMENT

Week 8, Handling User Input and Events (OOP Approach)

Name: KIHARA WANDERI
Reg No: BIT/2024/58742
Course: Bachelor of Science in Information Technology
Unit: BIT4107, Mobile Application Development
Date: June 2025

Objective

To implement keyboard input handling and touch gesture detection in a mobile app using an Object-Oriented Programming (OOP) class-based approach.

Introduction

Mobile apps respond to user input through events, such as keyboard presses, taps, swipes, and long presses. In OOP, each input type is handled by a specific class. This keeps the code organized and reusable.

Event Types

Event Type	Trigger	Class Method	Output
KeyDown	Key pressed down	<code>key_pressed(key)</code>	Key Detected: A
KeyUp	Key released	<code>key_released(key)</code>	Key Released: A
Tap	Single finger touch	<code>tap()</code>	Tap detected
Swipe Left	Finger drags left	<code>swipe_left()</code>	Previous Page
Swipe Right	Finger drags right	<code>swipe_right()</code>	Next Page
Long Press	Finger held down	<code>long_press()</code>	Context Menu shown
Double Tap	Two quick taps	<code>double_tap()</code>	Item selected

Class-Based Event Handling

1. Basic Event Handler

```
class EventHandler:
    def on_key_press(self, key):
        print('Key Pressed:', key)
    def on_tap(self):
        print('Screen tapped')

handler = EventHandler()
handler.on_key_press('Enter')
```

```
handler.on_tap()
```

2. Keyboard Controller

```
class KeyboardController:
    def key_pressed(self, key):
        if key == 'Enter':
            print('Form Submitted')
        elif key == 'Left':
            print('Move Left')
        else:
            print('Key Detected:', key)

controller = KeyboardController()
controller.key_pressed('Enter')
```

3. Input Validation

```
class LoginForm:
    def validate_username(self, username):
        if len(username) < 3:
            return 'Username too short'
        return 'Valid Username'

form = LoginForm()
print(form.validate_username('John'))
```

4. Gesture Handler

```
class GestureHandler:
    def tap(self):
        print('Tap detected')
    def swipe_left(self):
        print('Previous Page')
    def swipe_right(self):
        print('Next Page')
    def long_press(self):
        print('Context Menu')
    def double_tap(self):
        print('Item Selected')

g = GestureHandler()
g.tap()
g.swipe_right()
```

5. Mini App — All Events Combined

```
class MobileApp:
    def keyboard_input(self, text):
```

```
        print('Input Received:', text)
    def tap_event(self):
        print('Button Clicked')
    def swipe_event(self):
        print('Page Changed')

app = MobileApp()
app.keyboard_input('Hello')
app.tap_event()
app.swipe_event()
```

Practical Task

Build a Flutter app (week8_events) that demonstrates:

- Keyboard input — a text field with real-time event logging and input validation
- Tap, Double Tap, Long Press — each detected by GestureDetector and logged
- Swipe Left / Right — direction detected from drag velocity and logged
- Event Log — a scrollable list showing all interactions in order

All logic is organized into separate classes: KeyboardController, GestureHandler, EventLogger

See Flutter code submission on GitHub (week8 folder).

Conclusion

Organizing event handling into specific classes makes mobile apps easier to build, test, and extend. Each class manages one responsibility—keyboard, gestures, or logging—following OOP principles.

End of Week 8 Assignment